

并行计算研究的 结构化表达与可视化

汇报人：待填写 学号：待填写

哈尔滨工业大学

汇报日期：待填写



- ① 问题与模型
- ② 实现与机制
- ③ 图表与评价
- ④ 讨论与结论

① 问题与模型

② 实现与机制

③ 图表与评价

④ 讨论与结论

研究问题：并行度如何转化为端到端收益？

从工作负载定义研究对象

- **输入规模**：固定问题大小，讨论 strong scaling。
- **执行语义**：并行实现与串行实现完成相同计算。
- **评价边界**：明确传输、计算、同步分别是否计入。

可回答的问题

给定并行比例 f ，增加执行资源 p 的理论收益是多少？哪些额外成本需要进入实测模型？

 f

可并行工作占串行总时间的比例

 p

理想并行资源数量

本示例的曲线与数值来自解析模型，变量不绑定具体 GPU。

让听众先知道优化对象、时间范围与判断标准，再进入实现细节。

模型：串行部分给出了加速比的上限

固定问题规模的理想模型

$$T_p = T_1 \left((1 - f) + \frac{f}{p} \right)$$

$$S(p) = \frac{T_1}{T_p} = \frac{1}{(1 - f) + f/p}$$

$$\lim_{p \rightarrow \infty} S(p) = \frac{1}{1 - f}$$

该模型假设并行部分均匀分配，且忽略通信、调度和同步的额外成本。

符号与适用条件

符号	含义
T_1	串行总时间
T_p	使用 p 个资源的理想时间
f	可并行时间比例, $0 \leq f \leq 1$
$S(p)$	Speedup, 相同工作量下的时间比

公式负责定义关系；符号表负责降低理解成

本；注记负责限定结论范围。

方法对照：把决策依据放在同一张表中

组织方式	适合的工作	需要关注的成本	主要验证指标
Thread-per-task	独立且短小的任务	单线程状态量、访存合并	Register、带宽、吞吐
Warp-per-task	任务内存在协作	Lane 利用率、shuffle 成本	Warp efficiency、延迟
Block-per-task	较大协作范围	SMEM 容量、同步、占用率	SMEM、barrier stall
Pipeline	可分阶段的批量任务	提交、依赖、批次尾部	端到端时间、阶段重叠

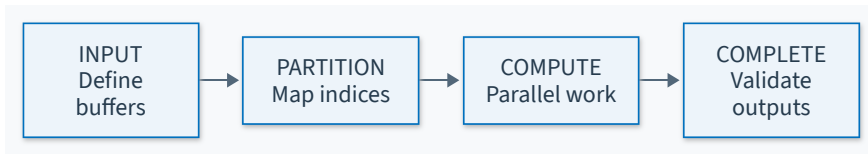
对照维度保持一致

每行给出同一种信息，明确映射方式与工作负载的联系。

结论需要实测支持

这些条目用于形成实验假设；具体配置的收益取决于输入与硬件。

流程图：用数据依赖组织实现说明



有向边表示依赖顺序；每个阶段的计时边界需要在实验中单独定义。

01 Input contract

输入类型、长度、布局 and 有效范围共同决定接口语义。

02 Parallel work

以明确的索引映射说明工作如何分配给线程。

03 Completion

输出有效性与完成时刻共同决定计时终点。

一张流程图对应一个抽象层次；并发关系与时间关系分别展开说明。

① 问题与模型

② 实现与机制

③ 图表与评价

④ 讨论与结论

IMPLEMENTATION

把算法中的一次操作，
落实为线程中的一次访问。

代码与机制：一个线程负责一个有效元素

CUDA: element-wise vector addition

```
1  __global__ void add(  
2      const float* a, const float* b,  
3      float* c, size_t n) {  
4      const size_t i =  
5          size_t(blockIdx.x) * blockDim.x  
6          + threadIdx.x;  
7      if (i < n) {  
8          c[i] = a[i] + b[i];  
9      }  
10 }
```

代码片段用于解释索引与访问关系。调用者负责分配有效设备缓冲区并检查执行状态。

三个阅读锚点

索引

$i = \text{blockIdx.x} \times B + \text{threadIdx.x}$

边界

$i < n$ 控制最后一个 block 的有效线程。

访问

相邻线程处理连续元素，形成相邻地址访问。

越界保护限定有效索引；输入输出缓冲区需满足契约。

访存布局：用地址表解释连续访问

同一个 Warp 的前四个 Lane，32-bit 元素

地址偏移 (Byte)	Lane 0	Lane 1	Lane 2	Lane 3
Stride = 1 element	0	4	8	12
Stride = 32 elements	0	128	256	384

计算规则

$$\text{offset}(\ell) = 4s\ell$$

ℓ 为 Lane 编号， s 为元素间距。地址表由公式直接计算。

与实测连接

使用内存事务、有效带宽与 Kernel 时间评估访问模式，并同时记录对齐条件。

内存事务数量还取决于参与 Lane、访问宽度、地址对齐和 GPU 架构。

机制说明：从局部计算走向完整执行路径

局部关系

- 连续地址访问影响访存事务。
- 线程状态影响 Register 需求。
- 协作范围影响同步方式。

局部机制用于解释观察结果。

完整路径

- 调用与数据准备进入计时边界。
- 传输与 Kernel 的依赖需要明确。
- 输出完成后才能计算端到端延迟。

最终指标覆盖用户实际等待的过程。

■ 当局部收益与总时间变化不一致时，使用阶段计时检查成本转移，并保持输入与配置一致。

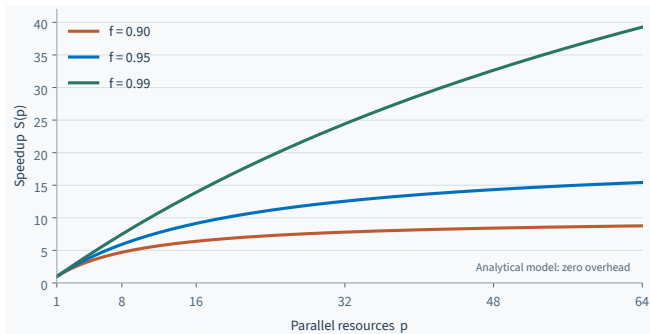
① 问题与模型

② 实现与机制

③ 图表与评价

④ 讨论与结论

Scaling curve: 串行比例限制资源扩展收益



先读趋势，再读边界

更高的并行比例带来更高的极限加速比。

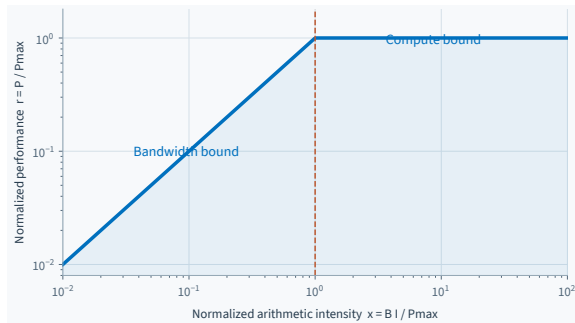
$20\times$

$f = 0.95$ 时, $p \rightarrow \infty$ 的理论上限

曲线由 Amdahl 模型计算，假设额外开销为零。

图题表达结论；坐标和图注定义数值含义；正文解释对研究问题的影响。

Roofline: 让两个约束出现在同一张图中



所有轴均为无量纲归一化量；拐点 $x = 1$ 对应带宽约束与计算约束相交。

归一化的解析关系

$$r = \min(1, x)$$

$$x = \frac{BI}{P_{\max}}, \quad r = \frac{P}{P_{\max}}$$

B : 带宽; I : 算术强度; $P_{\max}$: 计算上限。

图中仅展示模型约束。具体设备的上限需使用相应精度、指令类型和测量口径。

数值表：同一模型中的收益与效率

资源数量 p	归一化时间 T_p/T_1	Speedup $S(p)$	Efficiency $S(p)/p$
1	1.0000	1.000	100.00%
2	0.5250	1.905	95.24%
4	0.2875	3.478	86.96%
8	0.1688	5.926	74.07%
16	0.1094	9.143	57.14%
32	0.0797	12.549	39.22%

解析计算： $f = 0.95$ ， $T_p/T_1 = 0.05 + 0.95/p$ 。各列从未舍入数值独立取舍。

资源增加时，Speedup 与资源利用效率呈现不同变化；评价需要同时说明二者。

实验设计：把比较条件写到展示内容中

控制条件

条件	记录内容
输入	规模、分布、边界与随机种子
软件	Commit、Compiler、编译选项
硬件	GPU 型号、显存、运行状态
计时	Warm-up、同步位置、重复次数

输出证据

- 正确性：参考结果与边界输入检查。
- 性能：端到端延迟、吞吐与离散程度。
- 机制：阶段计时、资源与访问指标。

性能测量与机制分析分别记录运行配置，检查分析工具对执行时间的影响。

本页给出实验设计结构；在自己的报告中填入已执行的配置与结果文件位置。

① 问题与模型

② 实现与机制

③ 图表与评价

④ 讨论与结论

讨论：收益需要覆盖新增成本

在 Amdahl 模型中加入额外时间

$$T_p^{\text{actual}} = T_1 \left((1 - f) + \frac{f}{p} \right) + T_{\text{overhead}}(p)$$

调度成本

提交、任务划分和依赖管理，可能随资源或任务数量变化。

通信成本

数据移动、缓存行为和共享资源竞争，需要结合路径分析。

同步成本

负载不均与完成时间分布会影响整体结束时刻。

优化成立的条件：节约的有效计算时间足以覆盖新增的执行成本。

适用范围：把结论和外推条件对应起来

当前模型给出的结论	成立条件	进一步研究需要补充
串行比例限制 Speedup 资源增加降低理想时间	固定问题规模, f 已知 完美负载分配, 额外成本为零	工作量增长时的扩展模型 通信、调度与同步测量
Roofline 给出性能上界	B 、 I 、 $P_{\max}$ 定义一致	指令、访存与并发限制

■ 讨论页同时写清结论的支撑依据与适用范围, 让后续问题可以落到具体变量和实验上。

结论：问题、机制与证据形成完整链条

01

定义问题

说明输入、计算语义与评价边界，给出清晰的研究问题。

02

解释机制

用公式、代码和数据依赖说明映射方式及成本来源。

03

建立证据

让图表、实验条件与适用范围共同支撑最终判断。

每一页承担一个明确任务；每个关键判断都有可检查的依据。

感谢聆听 欢迎讨论

附录 A：中英混排、公式与代码字体

正文层次

研究对象：GPU 上的批量并行计算

Global memory、Warp shuffle、Register pressure 和 Occupancy 使用一致术语。

Source Sans 3 italic

Source Sans 3 bold / 思源黑体粗体

图注用于解释符号、条件与单位，重要信息随图表阅读。

数学与等宽字符

$$\mathbf{y} = \mathbf{Ax}, \quad \hat{a}_k = \sum_{j=0}^{n-1} a_j \omega^{jk}$$

```
1 size_t i = block * stride + lane;
2 // 00  11  {} [] ()  <= >= !=
```

JetBrains Mono: 0123456789

中文正文使用思源黑体；西文使用 Source Sans 3；
代码使用 JetBrains Mono。

附录 B：注记色块随文字高度自适应

简短注记

■ 计时包含设备端同步。

两行注记

■ 比较两种方法时，输入规模与计时范围保持一致，并在图表附近注明。

关键结论

让注记紧邻它所解释的内容。

多行注记

■ 当说明包含条件、适用范围或计量单位时，左侧色块按整个文本盒的高度绘制。色块宽度为 2 pt，文字左侧间距为 1.5 mm，适用于不同列宽和自然换行。

内容与层次

注记用于补充影响理解的条件；浅蓝结论区用于承载本页要点。

重要条件随图表展示；文献、许可与详细证据放入随附文档或附录。

附录 C：长标题能够自然换行， 并保持内容区域与页面边界清晰

阅读顺序明确的三线表

内容类型	展示方式	常用命令
概念与条件	小标题、正文、注记	<code>\subhead, \notebox</code>
公式与符号	双栏与符号表	<code>columns, tabularx</code>
数值与趋势	矢量图、图注、指标	<code>\fig, \metric</code>
过程与实现	Mermaid、代码片段	<code>\includegraphics, lstlisting</code>

常见版式保存在 `example/layouts/`；复制需要的 frame 即可使用。

附录 D：从起步文档组织自己的汇报

标题层级

- `\section`: 章节分组。
- `\subsection`: 可选子章节。
- `frame`: 一个页面及其标题。
- `\subhead`: 页内小标题。

文件入口

`slides/content.tex`: 正文。

`starter.tex`: 简洁起点。

`example/layouts/`: 版式。

`README.md`: 构建与版式说明。

使用 XeLaTeX 编译，并保留主题、字体与所引用资源的相对目录结构。